Telemedicine Platform

Production-Level System Design & A-to-Z Development Plan

Stack: Next.js + NestJS + Prisma + PostgreSQL + Better Auth + Redis

This document turns the provided Telemedicine Website proposal into a practical production-oriented architecture, database plan, booking flow, security model, and step-by-step implementation roadmap.

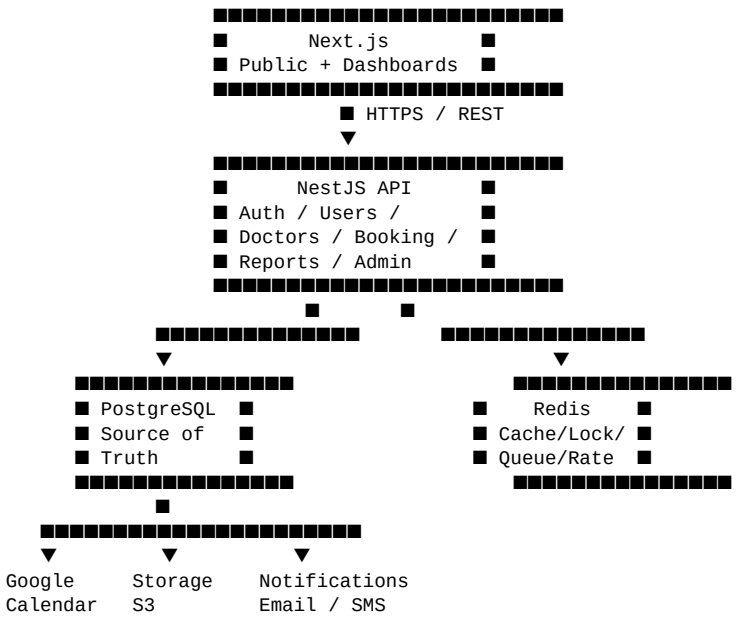
1. Project Overview

The platform connects patients with doctors for scheduled online consultations. The core roles are Doctor, Patient, and Super Admin. Patients can discover doctors, view availability, book appointments, upload medical reports, and join Google Meet. Doctors manage profiles and availability, view bookings and reports, and connect to patients. Super Admin manages doctors, patients, bookings, verification, and reporting.

2. Core Technology Stack

Layer	Technology	Purpose
Frontend	Next.js (App Router)	Public site, dashboards, SSR/CSR, UI
Backend	NestJS	REST API, business logic, authorization
ORM	Prisma	Database access and migrations
Database	PostgreSQL	Primary source of truth
Authentication	Better Auth	Registration, login, sessions, verification
Cache / Queue	Redis + BullMQ	Rate limiting, caching, jobs, reminders
File Storage	S3-compatible storage / Cloudinary	Medical reports and profile assets
Calendar / Video	Google OAuth + Calendar API + Google Meet	Calendar event and meeting creation
Notifications	Email provider + optional SMS	Booking confirmations and reminders

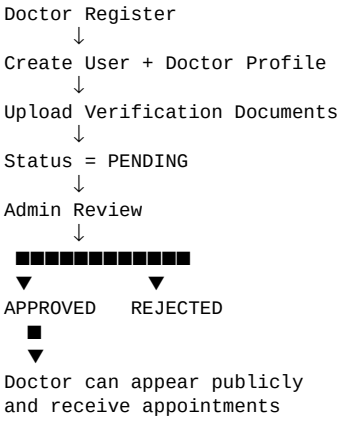
3. High-Level System Architecture



4. User Roles & Permissions

Role	Main Responsibilities
PATIENT	Search doctors, view slots, book/cancel appointments, upload reports, join meetings, review doctors
DOCTOR	Manage profile, submit verification, configure availability, view bookings/reports, join meetings
ADMIN	Verify/block doctors, manage patients, oversee bookings, view revenue and system reports

5. Doctor Verification Flow



Recommended DoctorVerification fields

id, doctorId, status, licenseNumber, licenseDocumentUrl, degreeDocumentUrl, submittedAt, reviewedAt, reviewedBy, rejectionReason.

6. Doctor Availability & Slot Engine

Doctors define recurring weekly availability. The backend generates concrete slots for a selected date and removes slots already occupied by appointments. Availability must be timezone-aware and should not be treated as simple frontend state.

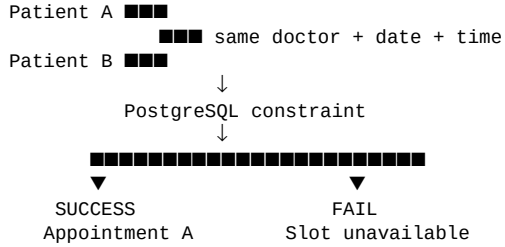
```
DoctorAvailability
id
doctorId
dayOfWeek
startTime
endTime
slotDuration
isActive

Example:
Sunday | 10:00 | 14:00 | 30 minutes

Generated:
10:00, 10:30, 11:00, 11:30,
12:00, 12:30, 13:00, 13:30
```

7. Appointment Engine — Core Business Logic

```
Patient
  ↓
Doctor Profile
  ↓
Select Date
  ↓
GET Available Slots
  ↓
Select Slot
  ↓
POST /appointments
  ↓
NestJS validation
  ■■ Authenticated patient?
  ■■ Doctor approved?
  ■■ Doctor available?
  ■■ Slot still free?
  ↓
PostgreSQL transaction
  ↓
Create Appointment
  ↓
Google Calendar / Meet
  ↓
Queue notifications
  ↓
Booking confirmed
```

8. Double-Booking Prevention

Double booking must be prevented at the backend/database level. Frontend slot state is not sufficient because two users can submit requests at nearly the same time.




```
Recommended:
UNIQUE(doctorId, appointmentDate, startTime)
+ database transaction
+ optional Redis short-lived distributed lock
```

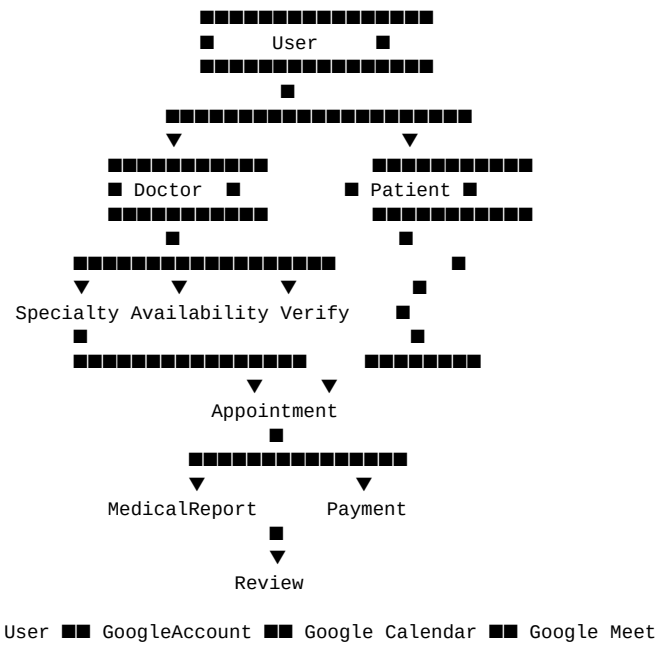
9. Appointment State Machine

```
PENDING
   ↓
CONFIRMED
   ■■■→ CANCELLED
   ■■■→ COMPLETED
   ■■■→ NO_SHOW

If online payment is introduced:
PENDING_PAYMENT → CONFIRMED
```

10. Database Design

Table	Purpose
User	Identity, authentication linkage, role, account status
Doctor	Professional profile, fee, experience, verification status
Patient	Patient-specific profile and personal details
Specialty	Medical departments/specialties
DoctorSpecialty	Doctor ↔ Specialty many-to-many relation
DoctorVerification	License/degree verification workflow
DoctorAvailability	Recurring weekly availability
DoctorDayOff	Specific date exceptions/days off
Appointment	Doctor-patient consultation booking
MedicalReport	Report metadata and secure storage references
GoogleAccount	Google OAuth/calendar integration
Notification	In-app/email notification records
Review	Completed appointment rating/review
Payment	Payment transaction records
AuditLog	Administrative/security activity history

11. Core Database Relationship



12. Recommended Core Model Fields

User — id, email, role, status, emailVerified, createdAt, updatedAt

Doctor — id, userId, bio, profileImage, experienceYears, consultationFee, qualification, verificationStatus, averageRating, totalReviews

Patient — id, userId, dateOfBirth, gender, address, emergencyContact

Specialty — id, name, slug, description, isActive

DoctorAvailability — id, doctorId, dayOfWeek, startTime, endTime, slotDuration, isActive

DoctorDayOff — id, doctorId, date, reason

Appointment — id, patientId, doctorId, appointmentDate, startTime, endTime, status, reason, meetingUrl, googleEventId, timestamps

MedicalReport — id, patientId, appointmentId, fileName, fileUrl, fileType, fileSize, uploadedAt

GoogleAccount — id, userId, googleUserId, encrypted tokens, tokenExpiresAt, scope

Notification — id, userId, type, title, message, readAt, createdAt

Review — id, appointmentId, patientId, doctorId, rating, comment, createdAt

Payment — id, appointmentId, amount, currency, provider, transactionId, status, timestamps

AuditLog — id, actorUserId, action, entityType, entityId, metadata, createdAt

13. Important Database Rules

• Doctor email/user identity should be unique through the authentication/user layer.

• Doctor + appointment date + start time must have a database-level uniqueness strategy.

• Use indexes for doctor/date, patient/date, appointment status, and doctor verification status.

• Use foreign keys with deliberate cascade/restrict behavior; do not blindly cascade medical or financial records.

• Store medical files in object storage, not as binary data inside PostgreSQL.

• Store only encrypted Google refresh/access tokens or a secure token reference.

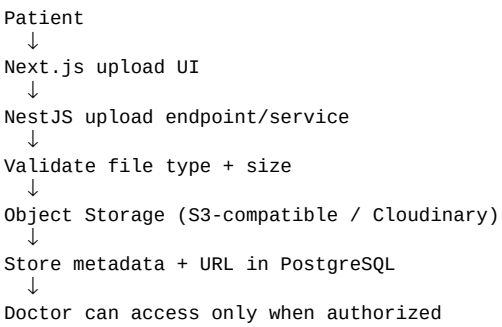
• Keep timestamps consistently managed and design appointment time handling around an explicit timezone strategy.

14. Google Calendar + Google Meet Flow

```
Doctor
  ↓
Connect Google account
  ↓
OAuth consent
  ↓
NestJS stores secure token information
  ↓
Patient books appointment
  ↓
NestJS creates Google Calendar event
  ↓
Conference / Google Meet generated
  ↓
meetingUrl + googleEventId saved
  ↓
Doctor + Patient receive confirmation
```

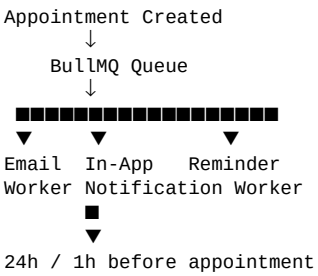
Important: Google login and Google Calendar authorization are different concerns. Calendar integration needs the appropriate Google OAuth scopes and should be handled server-side.

15. Medical Report Upload Architecture



Recommended initial allowed formats: PDF, JPG, JPEG, PNG. Add strict file-size limits, server-side validation, safe filenames, authorization checks, and private storage URLs where possible.

16. Notification & Background Job Architecture



17. Redis Responsibilities

• Distributed rate limiting for API protection.

• Short-lived caching for frequently requested doctor/slot data where safe.

• BullMQ job storage/coordination.

• Optional short-lived distributed locking around high-contention booking operations.

• Temporary state that does not need PostgreSQL durability.

Rule: PostgreSQL remains the source of truth for appointment ownership. Redis must not be the only protection against double booking.

18. NestJS Backend Module Structure

```
src/
├── auth/
├── users/
├── doctors/
├── patients/
├── specialties/
├── availability/
├── appointments/
├── google/
├── medical-reports/
├── notifications/
├── reviews/
├── payments/
├── admin/
├── uploads/
├── health/
├── common/
│   ├── guards/
│   ├── decorators/
│   ├── interceptors/
│   ├── filters/
│   ├── pipes/
│   ├── pagination/
```

```
■■■ prisma/
```

19. Next.js Application Structure

```
app/
■■■ (public)/
■   ■■■ page.tsx
■   ■■■ doctors/
■   ■■■ specialties/
■   ■■■ about/
■   ■■■ contact/
■■■ (auth)/
■   ■■■ login/
■   ■■■ register/
■■■ patient/
■   ■■■ dashboard/
■   ■■■ appointments/
■   ■■■ reports/
■   ■■■ profile/
■■■ doctor/
■   ■■■ dashboard/
■   ■■■ appointments/
■   ■■■ availability/
■   ■■■ patients/
■   ■■■ profile/
■■■ admin/
    ■■■ dashboard/
    ■■■ doctors/
    ■■■ patients/
    ■■■ appointments/
    ■■■ reports/
```

20. API Structure

Auth: POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me

Doctors: GET /doctors, GET /doctors/:id, PATCH /doctors/:id, GET /doctors/:id/availability

Verification: POST /doctors/verification, GET /admin/doctors/pending, PATCH /admin/doctors/:id/approve, PATCH /admin/doctors/:id/reject

Availability: POST /doctor/availability, GET /doctor/availability, PATCH /doctor/availability/:id, DELETE /doctor/availability/:id

Appointments: POST /appointments, GET /appointments, GET /appointments/:id, PATCH /appointments/:id/cancel, PATCH /appointments/:id/complete

21. Authorization Model

Use two layers: **RBAC** (role-based access control) and **resource ownership**. For example, a patient may have permission to cancel appointments generally, but the backend must also verify that the appointment belongs to that patient. Likewise, a doctor must only access reports and appointments they are authorized to see.

22. Security Checklist

• Better Auth session/authentication and email verification.

• Role guards for PATIENT, DOCTOR, ADMIN.

• Ownership checks for appointments, reports, and profiles.

• Global validation with DTOs/pipes.

• Global exception filter and consistent response format.

• Helmet/security headers and strict CORS configuration.

• Redis-based distributed rate limiting.

• Secure secret management through environment variables.

• Medical file type/size validation and private storage.

• Encrypted handling of Google OAuth tokens.

• Audit logs for sensitive admin actions.

• Payment webhook verification on the backend when payments are introduced.

23. Patient Dashboard

```
Patient Dashboard
■■■ Overview
■■■ Find Doctor
■■■ My Appointments
■   ■■■ Upcoming
■   ■■■ Completed
■   ■■■ Cancelled
■■■ Medical Reports
■■■ Previous Doctors
■■■ Profile
■■■ Notifications
```

24. Doctor Dashboard

```
Doctor Dashboard
■■■ Overview
■   ■■■ Today's Appointments
■   ■■■ Upcoming Appointments
```

```
■■■ Appointments
■■■ Availability
■   ■■■ Weekly Schedule
■   ■■■ Slot Duration
■   ■■■ Days Off
■■■ Patients
■   ■■■ Reports / History
■■■ Google Calendar
■■■ Profile
■■■ Verification
```

25. Admin Dashboard

```
Admin Dashboard
■■■ Overview
■   ■■■ Total Doctors
■   ■■■ Total Patients
■   ■■■ Today's Bookings
■   ■■■ Revenue
■■■ Doctor Management
■   ■■■ Pending Verification
■   ■■■ Approve
■   ■■■ Reject
■   ■■■ Block
■■■ Patient Management
■■■ Appointments
■■■ Payments
■■■ Reviews
■■■ Reports
■■■ Settings
```

26. A-to-Z Development Roadmap

Phase 0 — Requirement Freeze — Finalize roles, booking duration, cancellation policy, doctor verification requirements, reports, Google integration, notifications, and payment scope.

Phase 1 — Architecture Setup — Create Next.js and NestJS apps, PostgreSQL, Prisma, Redis, Docker, environment configuration, linting, formatting, and base project conventions.

Phase 2 — Authentication — Better Auth, registration, login, email verification, sessions, current-user handling, RBAC.

Phase 3 — User Management — Doctor/patient profiles, admin user management, active/blocked states.

Phase 4 — Doctor Verification — Verification profile, document upload, pending queue, admin approval/rejection/blocking.

Phase 5 — Specialties — Specialty CRUD, doctor assignment, public specialty pages and filtering.

Phase 6 — Availability Engine — Weekly schedule, slot duration, day-off exceptions, date-specific slot generation.

Phase 7 — Appointment Engine — Booking transaction, unique constraints, concurrency protection, appointment states, cancellation/completion.

Phase 8 — Google Integration — Google OAuth, Calendar permission, event creation, Meet URL generation, token refresh.

Phase 9 — Medical Reports — Secure upload, object storage, metadata, access control, download/view flow.

Phase 10 — Notifications — BullMQ, email worker, in-app notifications, appointment reminders.

Phase 11 — Dashboards — Patient, doctor, and admin dashboards with production UX and responsive behavior.

Phase 12 — Reviews — Only allow reviews for eligible/completed appointments; calculate doctor ratings.

Phase 13 — Payments — bKash/Nagad/Card, payment records, backend webhook verification, payment states.

Phase 14 — Testing — Unit, integration, E2E, especially concurrent booking tests.

Phase 15 — Production Hardening — Rate limits, caching, monitoring, logging, audit trails, backups, Docker, deployment, CI/CD.

27. Booking Flow — Master Diagram

```
Visitor
  ↓
Find Doctor
  ↓
Doctor Profile
  ↓
Available Slots
  ↓
Login / Register
  ↓
Optional Report Upload
  ↓
Confirm Booking
  ↓
NestJS
  ■■ Authentication
  ■■ Doctor approved?
  ■■ Availability
  ■■ Slot still available?
  ↓
PostgreSQL Transaction
  ↓
Appointment Created
  ↓
Google Calendar + Meet
  ↓
Notification Queue
  ■■ Doctor
```

```
  ■■ Patient
  ↓
Scheduled Consultation
  ↓
COMPLETED
  ↓
Review (optional)
```

28. MVP vs Production V1

MVP	Production V1
Auth	Auth + advanced security
Doctor profiles	Doctor verification + audit
Specialties	Advanced search/filter
Availability	Availability + day-off exceptions
Appointments	Concurrency-safe booking
Google Meet	Calendar sync + token lifecycle
Reports	Private object storage + access control
Notifications	Queue-based notifications + reminders
Dashboards	Analytics + monitoring

29. Final Architecture Principles

• Next.js is the UI/client application; NestJS owns business rules.

• PostgreSQL is the source of truth for durable business data.

• Prisma is the data-access layer, not the business-logic layer.

• Booking correctness is enforced at the database/backend level.

• Redis improves scalability and coordination but does not replace PostgreSQL.

• Medical documents are stored outside PostgreSQL with strict authorization.

• Google Calendar/Meet integration is isolated in its own module.

• Long-running work belongs in background queues.

• RBAC and resource ownership are both required.

• Build the system phase-by-phase; do not implement every feature at once.

Recommended next implementation step: freeze the requirements, then create the complete ERD and final Prisma schema before writing the appointment module. The database design should explicitly define relationships, indexes, unique constraints, enums, timestamps, and delete/cascade behavior.